

ESPRIT

École Supérieure Privée d'Ingénierie et de Technologies

Rapport de Stage

Stage d'immersion en entreprise

Réalisé au sein de

Lanterns-studios

Présenté par

Fadi Saidi

Étudiant en Esprit — 3ème année

Sous la supervision de

M. Oussema Ben Meriem

Responsable Département Développement

Période :

17 juillet 2025 — 17 septembre 2025

Remerciements

Je veux exprimer ici mes plus sincères remerciements à M. Oussema Ben Meriem, chargé du département développement au sein de l'entreprise Lanterns-studios, pour m'avoir accueilli au sein de son équipe et accompagné tout au long de ce stage.

Je veux également remercier l'ensemble de l'équipe de l'entreprise Lanterns-studios pour leur bienveillance, leur envie de transmettre leurs connaissances et l'ambiance de travail qu'ils ont su créer au sein de l'entreprise.

Enfin, je veux remercier ESPRIT pour la formation solide qu'ils m'ont transmise, me permettant ainsi d'approcher ce stage avec les connaissances techniques nécessaires, et ma famille pour son indéfectible soutien.

Table des matières

1 Introduction

- 1.1 Présentation de Lanterns-studios
- 1.2 Organigramme du studio
- 1.3 Contexte du projet
- 1.4 Problématique et solution proposée

2 Analyse et spécification des besoins

- 2.1 Introduction
- 2.2 Étude de l'existant
- 2.3 Besoins fonctionnels
- 2.4 Besoins non fonctionnels

3 Conception et architecture du système

- 3.1 Introduction⁹
- 3.2 Architecture générale
- 3.3 Architecture logique
- 3.4 Diagramme de cas d'utilisation
- 3.5 Diagramme de séquence
- 3.6 Stack technologique

4 Réalisation¹²

- 4.1 Mise en place de l'environnement
- 4.2 Système d'inventaire — Item Stacking
- 4.3 Système d'IA — Patrouille et détection
- 4.4 Intégration audio dynamique
- 4.5 Système de sauvegarde
- 4.6 Système de quêtes

5 Défis et améliorations futures

- 5.1 Difficultés rencontrées
- 5.2 Pistes d'amélioration

6 Conclusion

Chapitre 1 — Introduction

1.1 Présentation de Lanterns-studios

Lanterns-studios est un studio de développement de jeux vidéo indépendant spécialisé dans la conception d'expériences immersives sur PC et console. Le studio réunit une équipe pluridisciplinaire composée de programmeurs, de game designers et d'artistes 3D, travaillant conjointement sur des projets à fort potentiel narratif et technique.

Le département développement, au sein duquel j'ai effectué mon stage, est responsable de l'intégralité de la logique gameplay, de la programmation des systèmes de jeu en C++ et Blueprint, ainsi que de l'intégration de tous les éléments techniques du projet sous Unreal Engine 5.

1.2 Organigramme du studio

L'organisation de Lanterns-studios s'articule autour de plusieurs pôles complémentaires. Le tableau suivant représente la structure simplifiée du studio :

Direction Générale — Lanterns-studios		
DépartementDéveloppement (C++ / Blueprint / IA)	DépartementArt & Design (3D / Concept / UI)	DépartementProduction (QA / Planning / PM)

J'ai été affecté au département développement pour effectuer mon stage.

1.3 Contexte du projet

Ce stage d'immersion en entreprise s'est déroulé du 17 juillet au 17 septembre 2025, dans le cadre de ma troisième année à ESPRIT. J'ai intégré l'équipe de développement qui travaille sur un projet de jeu vidéo en cours de production sous Unreal Engine 5.4.

Dans ce contexte, j'ai été amené à contribuer à plusieurs systèmes gameplay du projet : le système d'inventaire, l'intelligence artificielle des PNJ, l'intégration audio, le système de sauvegarde et le système de quêtes. Ces contributions se sont inscrites dans les cycles de sprint hebdomadaires de l'équipe, avec suivi des tâches via Jira.

1.4 Problématique et solution proposée

1.4.1 Étude de l'état actuel

Comme tout projet de jeu vidéo en cours de développement actif, le projet de Lanterns-studios présentait plusieurs systèmes gameplay en cours d'implémentation ou nécessitant des améliorations. Parmi les problèmes identifiés lors de mon intégration :

- Le système d'inventaire ne gérait pas encore l'empilement d'objets identiques, obligeant le joueur à avoir une entrée distincte par objet, ce qui saturait rapidement l'interface.
- Les PNJ n'avaient pas de comportement d'IA cohérent : ils restaient statiques et ne réagissaient pas à la présence du joueur.
- Aucun mécanisme de sauvegarde de l'inventaire n'était implémenté, ce qui rendait la progression du joueur non persistante entre les sessions.
- Le système de quêtes ne disposait pas d'affichage HUD fonctionnel, et un bug causait la perte de progression au chargement d'une sauvegarde.

1.4.2 Solution proposée

Pour répondre à ces problématiques, j'ai, sous la supervision de mon encadrant, contribué au développement et à l'amélioration de ces systèmes en C++ et Blueprint. La solution apportée s'appuie sur les mécanismes natifs d'Unreal Engine 5 : Behavior Trees pour l'IA, DataTables pour la configuration des données, USaveGame pour la persistance, et UMG pour les interfaces utilisateur.

Chapitre 2 — Analyse et spécification des besoins

2.1 Introduction

Cette phase est cruciale dans l'exécution du projet car elle permet de déterminer ce dont nous avons besoin pour développer ou améliorer les systèmes ciblés. Elle facilite également la détermination des fonctionnalités nécessaires avec une approche adaptée au contexte de développement de jeu vidéo.

2.2 Étude de l'existant

Avant de commencer à contribuer, j'ai consacré du temps à l'analyse du code source existant et de la documentation interne du projet. Le projet utilise une architecture orientée composants, standard dans les projets Unreal Engine : chaque fonctionnalité gameplay est encapsulée dans un UActorComponent attaché aux acteurs concernés.

Les systèmes existants présentaient les caractéristiques suivantes :

- Inventaire : présent mais sans gestion de pile — chaque objet occupait une entrée indépendante.
- IA : les PNJ disposaient d'un AI Controller basique sans comportement défini.
- Audio : assets MetaSounds présents dans le projet mais non intégrés à la logique gameplay.
- Sauvegarde : structure USaveGame existante mais ne couvrant pas l'inventaire du joueur.
- Quêtes : structure de données présente mais HUD non connecté et bug de chargement connu.

2.3 Besoins fonctionnels

Les fonctionnalités attendues au terme du stage sont les suivantes :

N°	Système	Besoin fonctionnel
F 01	Inventaire	Permettre l'empilement d'objets identiques avec une limite configurable par type.
F 02	Inventaire	Afficher le compteur de pile sur chaque icône d'objet dans le widget UMG.
F 03	IA	Faire patrouiller les PNJ entre une série de waypoints définis dans le niveau.
F 04	IA	Permettre aux PNJ de détecter le joueur dans un cône de vision et passer en état d'alerte.
F 05	Audio	Déclencher des sons d'alerte lorsqu'un PNJ détecte le joueur.
F 06	Sauvegarde	Persister l'état de l'inventaire du joueur entre les sessions de jeu.
F 07	Quêtes	Afficher les objectifs de quête actifs sur le HUD du joueur en temps réel.
F 08	Quêtes	Corriger le bug de réinitialisation de la progression des quêtes au chargement.

2.4 Besoins non fonctionnels

Performance

- Les mises à jour de l'IA ne doivent pas s'exécuter à chaque frame pour préserver les performances sur le matériel cible.
- Le temps de réponse des interactions inventaire et quêtes doit être imperceptible pour le joueur (< 16ms).

Maintenabilité

- Le code doit respecter les conventions de nommage et d'architecture internes du studio.
- Les systèmes développés doivent être documentés dans le wiki Confluence du projet.

Stabilité

- Toute contribution passe par une pull request soumise à code review avant d'être mergée sur la branche principale.
- Les nouvelles fonctionnalités ne doivent pas introduire de régressions sur les systèmes existants.

Ergonomie

- Les interfaces UMG (inventaire, HUD quêtes, indicateur d'alerte) doivent être claires et lisibles sans surcharger l'écran.

Chapitre 3 — Conception et architecture du système

3.1 Introduction

Ce chapitre présente la conception architecturale du projet et les choix technologiques effectués. Nous débutons par une vue d'ensemble de l'architecture système, suivie des diagrammes comportementaux, et concluons par la stack technologique utilisée.

3.2 Architecture générale

Le projet suit une architecture client orientée composants, caractéristique des projets Unreal Engine. L'ensemble de la logique s'exécute localement et s'organise en trois couches distinctes :

- Couche Présentation : widgets UMG (inventaire, HUD, indicateur d'alerte) construits en Blueprint et liés au C++.
- Couche Logique Gameplay : composants C++ (UInventoryComponent, UQuestManager, UAIController) implémentant les règles du jeu.
- Couche Données : DataTables CSV (items, quêtes), fichiers USaveGame pour la persistance, assets MetaSounds pour l'audio.

3.3 Architecture logique

L'architecture logique du projet illustre les flux de données entre les différentes couches. Les composants C++ exposent des delegates (FOnInventoryUpdated, FOnQuestUpdated, FOnAlertTriggered) auxquels les widgets Blueprint s'abonnent. Cette approche découple la logique de la présentation et simplifie la maintenance du code.

COUCHE PRÉSENTATION			
Widget UMGInventaire	HUD Quêtes& Alerte IA		BlueprintUI Events
COUCHE LOGIQUE GAMEPLAY — C++			
UInventoryComponent Inventaire	AAIControllerBTTask / BTDecorator IA	UQuestManagerDelegate broadcast Quêtes	USaveGamePersistance Sauvegarde
COUCHE DONNÉES			
DataTableItems / Quêtes	USaveGameFichier .sav		MetaSoundsAssets audio

3.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous capture les exigences fonctionnelles du point de vue des acteurs. Les acteurs principaux sont le Joueur et le PNJ (personnage non joueur).

Acteur	Cas d'utilisation	Système concerné
Joueur	Ramasser et empiler des objets	Inventaire

Joueur	Consulter l'inventaire	Inventaire
Joueur	Sauvegarder / charger la progression	Sauvegarde
Joueur	Consulter les objectifs de quête	Quêtes / HUD
PNJ	Patrouiller entre des waypoints	IA — Behavior Tree
PNJ	Détecter le joueur et déclencher une alerte	IA — Perception
Système	Déclencher un son d'alerte	Audio — MetaSounds

3.5 Diagramme de séquence — Détection IA

Le diagramme de séquence suivant illustre le flux d'événements lors de la détection du joueur par un PNJ, depuis la perception jusqu'au déclenchement du son d'alerte :

#	Émetteur	Récepteur	Message / Action
1	PNJ (Perception)	→ BTDecorator	Vérification ligne de vue (CanSeePlayer ?)
2	BTDecorator	→ Blackboard	Écriture : LastKnownPlayerLocation
3	Blackboard	→ AI Controller	Changement d'état : Alerte (GameplayTag)
4	AI Controller	→ Game Mode	Broadcast : FOnAlertTriggered
5	Game Mode	→ MetaSounds	Déclenchement du son d'alerte
6	AI Controller	→ HUD Widget	Mise à jour de l'indicateur d'alerte visuel

3.6 Stack technologique

Unreal Engine 5.4

Moteur de jeu principal du projet. Fournit l'ensemble des systèmes de base : rendu, physique, audio, IA, réseau et interface utilisateur.

C++ (MSVC)

Langage principal utilisé pour l'implémentation de la logique gameplay. Offre des performances optimales et un accès complet aux APIs d'Unreal Engine.

Blueprint (Unreal Engine)

Système de scripting visuel d'Unreal Engine, utilisé pour les widgets UMG, le prototypage rapide et la liaison entre la logique C++ et l'interface utilisateur.

Visual Studio 2022

Environnement de développement intégré (IDE) utilisé pour l'édition, la compilation et le débogage du code C++.

Perforce (P4V)

Système de gestion de versions utilisé pour les assets volumineux (textures, modèles 3D, sons) qui ne sont pas adaptés à Git.

Git / Jira

Git pour la gestion du code source, Jira pour le suivi des tâches et la gestion de sprint.

Chapitre 4 — Réalisation

4.1 Mise en place de l'environnement

La première semaine du stage a été consacrée à la mise en place de l'environnement de développement. Avec l'aide d'un développeur senior, j'ai installé Unreal Engine 5.4, Visual Studio 2022 et le client Perforce (P4V). La première compilation du projet a nécessité plusieurs heures compte tenu du volume important du code source.

Cette phase m'a permis de comprendre la structure du projet (dossiers Source/, Content/, Config/) et de prendre en main les outils de l'équipe : tableau Jira pour le suivi des tâches, wiki Confluence pour la documentation, et processus de pull request pour la livraison du code. J'ai également participé à mes premiers stand-ups quotidiens et découvert le rythme des sprints.

4.2 Système d'inventaire — Item Stacking

Ma première tâche solo a consisté à étendre le système d'inventaire existant pour prendre en charge l'empilement d'objets identiques (item stacking). La logique de comptage a été intégrée dans le composant `UInventoryComponent` en C++. Les tailles de pile maximales par type d'objet sont externalisées dans une `DataTable` (`ItemDataTable`) éditables directement dans l'éditeur Unreal par l'équipe design, sans modification du code source.

Le widget UMG affichant l'inventaire a également été mis à jour pour afficher le compteur de pile sur chaque icône d'objet via une liaison Blueprint. Cette tâche m'a confronté à mes premières difficultés avec la structure de données existante. Après discussion avec le lead, j'ai adopté une approche cohérente avec l'architecture en place. La fonctionnalité a été livrée après une code review et la correction des cas limites identifiés (pile pleine, objet non-empilable).

4.3 Système d'IA — Patrouille et détection

Le travail sur l'intelligence artificielle des PNJ s'est déroulé en deux phases successives, en utilisant le framework Behavior Tree natif d'Unreal Engine.

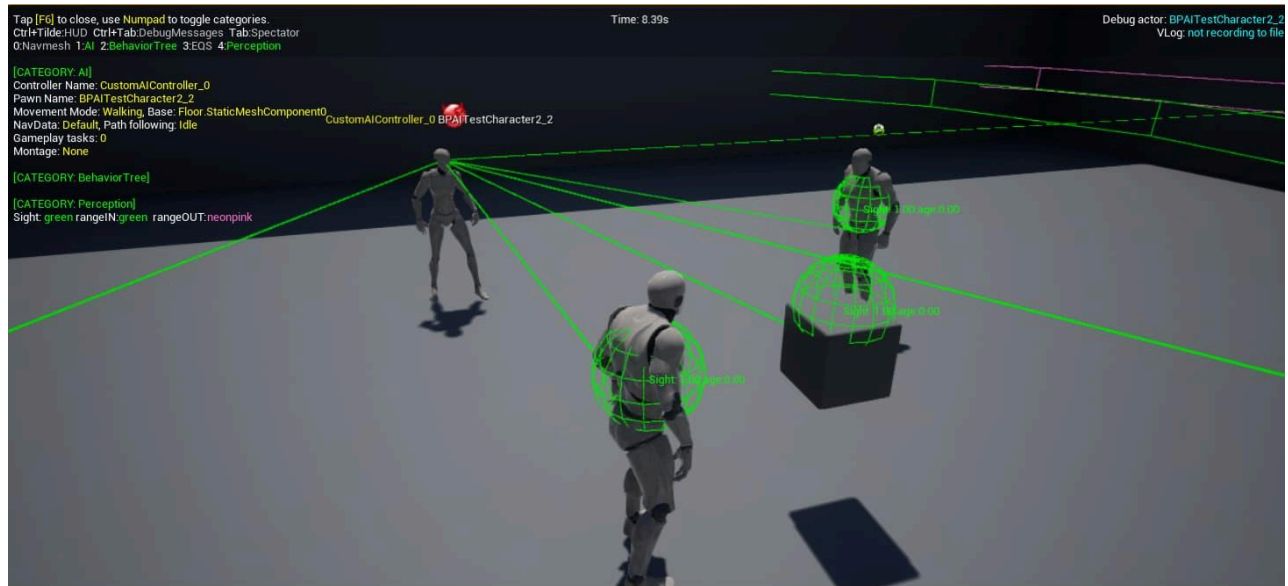
4.3.1 Patrouille

J'ai créé un nœud `BTTask` personnalisé (`BTTask_Patrol`) permettant aux PNJ de se déplacer entre une série de waypoints définis dans le niveau via le système de navigation `NavMesh`. Un bug lié à la fréquence d'images a été identifié et corrigé : le PNJ sautait des waypoints à haute fréquence d'images. La correction a consisté à appliquer une condition de fin de tâche correctement configurée et indépendante du delta-time.

4.3.2 Détection du joueur

En s'appuyant sur le système de perception d'Unreal Engine (`UAI_Sense_Sight`), j'ai implémenté un `BTDecorator` (`BTDecorator_CanSeePlayer`) permettant au PNJ de détecter le joueur dans un cône de vision paramétrable (angle, distance). Les états résultants sont gérés via des variables Blackboard (`LastKnownPlayerLocation`, `AlertLevel`) et des `GameplayTags` pour le changement d'état de l'AI Controller. Un bug où le PNJ restait bloqué en état d'alerte après avoir perdu de vue le joueur a également été corrigé.

Un indicateur visuel d'alerte a été ajouté au HUD du joueur pour refléter en temps réel l'état de détection des PNJ, en liaison avec les variables Blackboard via un sous-système Blueprint.



4.4 Intégration audio dynamique

En parallèle du travail sur l'IA, j'ai contribué à l'intégration de cues audio dynamiques déclenchées par événements gameplay. Utilisant le système MetaSounds d'Unreal Engine 5, j'ai connecté des event dispatchers C++ définis sur le Game Mode aux assets audio correspondants via UAudioComponent.

Un bug de superposition de sons a été identifié lors des tests : plusieurs événements d'alerte se déclenchant simultanément entraînaient une saturation du mix audio. Le problème a été résolu en ajoutant une vérification de l'état du composant audio avant chaque déclenchement, et en implémentant une logique de priorité entre les sons.

4.5 Système de sauvegarde

J'ai étendu le système de sauvegarde existant pour inclure la persistance de l'état de l'inventaire du joueur entre les sessions. Le travail s'est appuyé sur la classe USaveGame d'Unreal Engine et les fonctions UGameplayStatics::SaveGameToSlot et LoadGameFromSlot.

Un bug de rechargement de l'inventaire a été rencontré lors de la phase de test : certains objets n'étaient pas correctement restaurés à leur quantité sauvegardée. Après investigation avec le lead développeur, le problème a été identifié comme un ordre d'initialisation incorrect entre le composant d'inventaire et la lecture du fichier de sauvegarde. La correction a consisté à différer le chargement de l'inventaire après l'initialisation complète du composant.

4.6 Système de quêtes

Pour le système de quêtes déjà présent dans le projet, j'ai apporté deux contributions principales :

4.6.1 Affichage HUD

J'ai modifié le widget Blueprint existant pour afficher les objectifs de quête actifs sur le HUD du joueur. La liaison entre le C++ et le widget s'effectue via un delegate broadcast (FOnQuestUpdated) émis par le composant QuestManager à chaque changement d'état d'une quête, permettant une mise à jour en temps réel sans couplage direct entre les systèmes.

4.6.2 Correction du bug de sauvegarde

Un bug critique signalé par l'équipe QA causait la réinitialisation de la progression des quêtes actives lors du chargement d'une sauvegarde. Après investigation avec le lead développeur, j'ai identifié et corrigé le problème : le gestionnaire de quêtes était initialisé avant la lecture du fichier de sauvegarde, écrasant les données chargées. La correction a consisté à réordonner les appels d'initialisation dans le Game Mode.

Chapitre 5 — Défis et améliorations futures

5.1 Difficultés rencontrées

Prise en main d'une large base de code existante

L'un des premiers défis a été la compréhension d'une base de code de grande taille, avec ses propres conventions et abstractions. La lecture du code source du moteur Unreal a été indispensable pour comprendre le comportement des classes de base avant de les étendre.

Bugs liés à l'environnement et à l'ordre d'initialisation

Plusieurs bugs rencontrés n'étaient pas directement liés à mon code, mais à une mauvaise configuration de l'environnement (NavMesh pour la navigation IA) ou à un ordre d'initialisation incorrect (sauvegarde, quêtes). Ces situations m'ont appris l'importance de valider l'environnement avant de chercher des erreurs dans la logique applicative.

Gestion du temps et communication

Travailler dans un contexte de sprint avec des deadlines réelles a nécessité d'apprendre à estimer correctement l'effort requis pour chaque tâche et à communiquer rapidement en cas de blocage, plutôt que de rester bloqué seul de manière prolongée.

5.2 Pistes d'amélioration

Améliorations des capacités IA

- Implémentation d'une IA d'investigation : après la perte de vue du joueur, le PNJ se rend à la dernière position connue avant de reprendre sa patrouille.
- Ajout d'un système de communication entre PNJ : un PNJ en alerte peut notifier les PNJ voisins.

Améliorations du système de sauvegarde

- Ajout d'un système de sauvegarde automatique à intervalles réguliers ou lors d'événements clés du jeu.
- Versionnement des fichiers de sauvegarde pour gérer les mises à jour du jeu sans corrompre les sauvegardes existantes.

Améliorations techniques

- Mise en place de tests automatisés (Unreal Automation Framework) pour valider les systèmes d'inventaire et de sauvegarde.
- Profilage approfondi avec Unreal Insights pour identifier d'autres opportunités d'optimisation CPU.

Chapitre 6 — Conclusion

6.1 Bilan du projet

Ce stage d'immersion en entreprise chez Lanterns-studios m'a permis de contribuer activement au développement d'un projet de jeu vidéo professionnel sous Unreal Engine 5.4. Au fil de ces deux mois, j'ai participé à l'implémentation et à l'amélioration de cinq systèmes gameplay distincts : l'inventaire, l'intelligence artificielle des PNJ, l'intégration audio, la sauvegarde et les quêtes.

Ces contributions m'ont permis de vivre l'intégralité du cycle de développement d'une fonctionnalité en contexte professionnel : analyse de l'existant, conception, implémentation, test, code review et livraison. J'ai également pris part aux processus agiles de l'équipe (stand-ups, sprints, rétrospectives), ce qui m'a donné une vision concrète du fonctionnement d'une équipe de développement de jeux vidéo.

6.2 Réalisations techniques

Les principales réalisations techniques de ce stage sont les suivantes :

- Maîtrise pratique de l'API C++ d'Unreal Engine 5 : composants, delegates, Behavior Tree, DataTables, UMG, MetaSounds, USaveGame.
- Implémentation d'un système d'IA complet (patrouille + détection) via le framework Behavior Tree natif d'Unreal.
- Intégration d'un pipeline audio dynamique piloté par les événements gameplay via event dispatchers C++.
- Résolution de bugs techniques liés à des problèmes d'ordre d'initialisation, de configuration NavMesh et de superposition audio.

6.3 Compétences acquises

Au-delà des compétences techniques, ce stage m'a permis de développer des réflexes professionnels essentiels : lire du code existant avant de modifier, tester son travail avant de soumettre une pull request, demander de l'aide rapidement plutôt que de rester bloqué, et documenter ses contributions pour assurer la continuité du projet.

Ce stage m'a confirmé mon intérêt pour le développement de systèmes temps réel et la programmation de jeux vidéo. Les notions d'architecture logicielle, de performance et de collaboration en équipe que j'ai approfondies ici constitueront des atouts précieux pour la suite de ma formation à ESPRIT et pour mes futurs projets professionnels.